SIMATS ENGINEERING

Department of Information Technology

DSA01 – Object-Oriented Programming with C++

ASSIGNMENT 1 – CO1

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs.

Student Name	**Ragul S**	**Register Number**	**1911260201**
Department / Section	Btech CSE	**Academic Year / Semester**	1 st year
Faculty Name	Amirtha	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	06/11/2026	**Date of Evaluation**	
GitHub Repository	DSA01-<RegisterNumber>-Assignment1	**Compiler**	g++ 13.3.0, -std=c++17 -Wall -Wextra

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Object-Oriented Analysis	4		
2	Tokens, Keywords, Identifiers and Constants	3		
3	Data Types and Compatibility	4		
4	Implicit Conversions	4		
5	Operators and Precedence	4		
6	Control Structures	4		
7	Procedural vs Object-Oriented	4		
8	Input and Output	4		
9	Implementation and Testing	4		
10	Expectation vs Execution	4		
11	Final Recommendation	4		
12A	Re-evaluate: Impact Analysis	2		
12B	Re-evaluate: Evaluation of Original Design	2		
12C	Re-evaluate: Adaptation or Redesign	2		
12D	Re-evaluate: Evidence-Based Defence	2		
	TOTAL	50		Percentage:

OVERALL REMARKS

Student Signature	**Faculty Signature**	**Program Director**

OBJECT-ORIENTED PROGRAMMING WITH C++
ASSIGNMENT 1 – CO1

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: **06/11/2026**
Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System (Total Marks - 50)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (Task01, Task02 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version, the exact g++ command used to compile, the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. A single upload on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report.

GitHub Repository Link: **https://github.com/Ragul298/C--1911260201.git**

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Namma Metro Smart-Card Fare System

A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs 20 or Rs 30 according to the farthest zone travelled, with a 20% surcharge during peak hours (8–10 AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession.

The first version of the gate software was written in a hurry. Some fares are off by a paisa, the peak check accepts 10 AM as peak and 8 PM as off-peak inconsistently, a negative balance once appeared on a card, and the fare values are typed in several places.

The number of journeys processed may grow: 1 station (20,000 journeys a day) → 1 line (500,000 a day) → whole network (3,000,000 a day).

You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants, operators and control structures, implement the fare logic and validate it, so that every fare and balance is exact.

Design Challenge:

The metro is adding a fourth zone, a weekend discount and monthly passes. Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects **4 Marks**

Analyse the fare problem from the object-oriented point of view.

Discuss:

- The objects in the problem (card, passenger, journey, zone, fare table) and the data and behaviour of each
- How the same problem would be written in a procedural style, and what changes in an object-oriented style
- The basic principles of OOP – encapsulation, abstraction, inheritance and polymorphism – and where each could appear in this system
- The rules every correct journey must satisfy (for example: balance never below Rs 50 after a journey)
- You are expected to justify each object and rule, not merely list names.

Alternative Concept and Complexity Analysis

If the fare system were written in a purely procedural style (global variables and free functions) instead of SmartCard, Journey and FareTable classes, what would the solution be? Analyse whether the time per journey or the memory per card changes (both are O(1)), and compare how much code must be changed when one fare rule changes.

Ans:

Object-Oriented Analysis of the Metro Fare System

1. **Understanding the problem**

Problems in the first gate software: (i) fares off by a paisa, (ii) 10 AM and 8 PM treated inconsistently, (iii) a card went negative, (iv) fare values typed in many places. All four come from one cause: nothing owns the money and the rules. So the design gives each item one owner, keeps money in whole numbers, and keeps each number in one place.

Assumptions (the scenario does not state them):

- **A1 – Concession:** students and seniors pay 50% of the base fare; the surcharge is added after the concession.
- **A2 – Zone:** the fare depends only on the farthest zone reached.
- **A3 – Peak:** entry time from 08:00 up to 09:59, or 17:00 up to 19:59. So 8:00 and 17:00 are peak; 10:00 and 20:00 are off-peak.
- **A4 – Money:** stored as whole paise in a long long; percentages use integer arithmetic, rounded half-up.
- **A5 – Balance rule:** a journey is allowed only if balance − fare ≥ 5000 paise (Rs 50). Recharge must be positive.

Table 1: Objects, their data and behaviour, and why each one exists

Object	Data (attributes)	Behaviour (methods)	Why it is an object
SmartCard	balance_ (long long, paise, private)	balance(), canTravel(f), deduct(f), recharge(a)	It owns the money. Only the card may change its balance, so the Rs 50 rule can be enforced in one place.
Passenger	name, category (Regular / Student / Senior)	isConcession()	The concession depends on who travels, not on the journey.
Journey	farthest zone, hour, minute	isPeak()	One journey is one unit of work at the gate; it carries the time that decides the surcharge.
Zone	Inner, Middle, Outer (enum class)	– (a value type)	A zone is one of a fixed set of values; a type of its own stops it being mixed with ordinary ints.
FareTable	BASE_PAISE[], PEAK_SURCHARGE_PERCENT, CONCESSION_PERCENT, MIN_BALANCE_PAISE	fare(journey, concession)	All fare values and the fare formula live here, so a revision is made in one place.

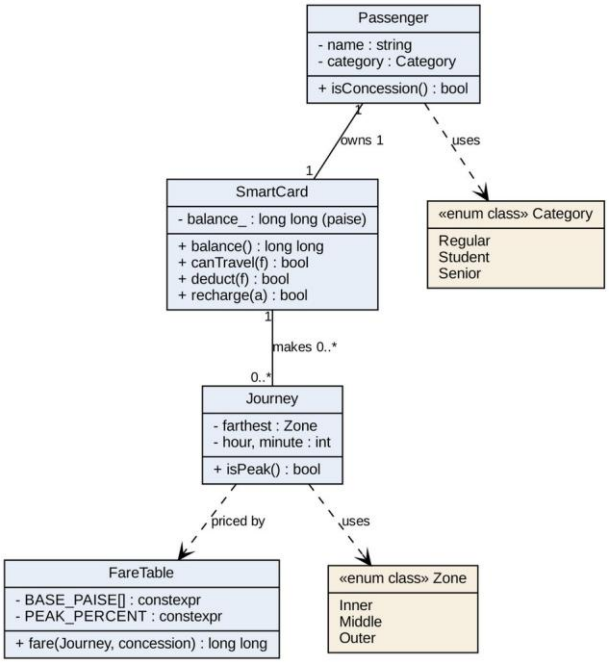


Figure 1: UML class diagram of the original design (solid line = association, dashed arrow = uses)

2. Procedural style versus object-oriented style

In a **procedural** program, balance and fares are global variables and the functions are separate, so the Rs 50 rule works only if every caller remembers to call canTravel() before deduct(). In the **object-oriented** program the card keeps balance_ private and deduct() does the check itself. The rule moves from the caller's memory to the card's code. Both versions are run in Task 7.

Table 2: OOP principles and where each appears in this system

Principle	Where it appears	Benefit here
Encapsulation	SmartCard hides balance_; FareTable hides the constants	No code outside the card can create a negative or unchecked balance.
Abstraction	The gate calls fare(journey) and card.deduct(f) without knowing how percentages or zones are handled	The gate code does not change when the fare formula changes.
Inheritance	Concession categories: StudentPassenger and SeniorPassenger can inherit from Passenger; a monthly Pass can inherit from a common FareRule base class (used in Task 12)	Shared data (name, card) is written once; only the concession differs.
Polymorphism	A base class DiscountRule with a virtual apply(fare) overridden by ConcessionRule, WeekendRule, PeakRule	New rules (weekend discount) are added without editing the gate loop.

Table 3: Rules every correct journey must satisfy

No.	Rule	Where it is enforced
R1	After the fare is deducted the balance must be at least Rs 50 (5000 paise); so it can never be negative	SmartCard::deduct() / canTravel()
R2	Base fare is Rs 10, 20 or 30 by farthest zone	FareTable::BASE_PAISE
R3	Peak is 08:00–09:59 and 17:00–19:59 only; the surcharge is exactly 20% of the (concession-adjusted) fare	Journey::isPeak(), FareTable
R4	Students and seniors pay 50% of the base fare (assumption A1)	FareTable::CONCESSION_PERCENT
R5	All money is in integer paise; no floating point is used for money	Type of every money variable
R6	A recharge must be positive; an unknown zone or invalid time is rejected, not guessed	Input validation (Task 8)
R7	Each fare constant is written once	constexpr in one header

3. Alternative concept: purely procedural design

Procedural solution: global balance, global fare arrays and free functions. Pseudocode:

```
ALGORITHM ProcessJourney_Procedural
INPUT  : zone, hour, minute          (global: balance, BASE[], MIN_BAL)
OUTPUT : gate decision, updated balance
BEGIN
   fare <- fareOf(zone, hour, minute)        // free function reads BASE[]
   IF balance - fare >= MIN_BAL THEN         // caller must remember this test
      balance <- balance - fare
      RETURN OPEN
   ELSE
      RETURN DENY
   END IF
END
```

Time. Both do the same few steps per journey (one lookup, up to two percentage calculations, one comparison, one subtraction). I timed 50 million journeys: procedural 234 ms, object-oriented 226 ms. The difference is noise, so both are **O(1)**.

Space. sizeof(long long) and sizeof(SmartCard) are both 8 bytes, so memory per card is **O(1)** in both.

Changing a rule. If the minimum balance changes, both designs edit one constant. But in the procedural program every function can touch balance, so all must be re-checked. In the OO program only SmartCard can, so the change stays in one class.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly 3 Marks

---

Define the fares, surcharge and limits so that a revision is made in one place only.

Discuss:

- The five kinds of tokens in your program (keywords, identifiers, constants, operators, punctuators) with examples
- #define, const and constexpr constants, and why constexpr is preferred for the fare table
- enum and enum class for zones, and why enum class is safer
- Naming rules and conventions for identifiers

Alternative Concept and Complexity Analysis

If the fares were written with #define macros instead of constexpr constants and an enum class, what would change? Compare type checking and scope, and analyse the memory used (a macro occupies no storage; a constexpr value usually does not either) and whether the running time is affected.

Ans:

Tokens, Keywords, Identifiers and Constants

1. **The five kinds of tokens**

I split two lines of my program into tokens: constexpr long long MIN_BALANCE_PAISE = 5000; and if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;.

Table 4: Token kinds with examples from the fare program

Token kind	Examples from my program	Remark
Keywords	constexpr, long, int, bool, enum, class, switch, case, break, if, else, return, for, const	Reserved by C++; cannot be used as names
Identifiers	Zone, BASE_PAISE, fare, canTravel, balance, MIN_BALANCE_PAISE, trips	Names chosen by me
Constants (literals)	5000, 1000, 20, 100, true, false, '.', "Denied (low balance)"	Integer, boolean, character and string literals
Operators	=, +=, *, /, %, -=, <<, ::, &&, ||, ?:, <, >=	Specify computations
Punctuators	{ } ( ) [ ] ; , :	Separate and group; carry no value

2. **#define, const and constexpr**

Table 5: Ways of naming a constant

Feature	#define	const	constexpr
Handled by	Preprocessor (text replacement)	Compiler	Compiler
Has a type	No	Yes	Yes

Feature	#define	const	constexpr
Obeys scope rules	No (global from the #define onward)	Yes	Yes
Value known at compile time	Yes (it is just text)	Only if initialised by a constant expression	Always (compiler checks)
Usable as array size / case label	Yes	Sometimes	Yes, guaranteed
Visible to the debugger	No	Yes	Yes

I chose **constexpr** because it is typed, checked at compile time and scoped. A macro is only text replacement. Example of the macro trap with #define FARE_INNER 500 + 500:

Output:

```
macro  FARE_INNER*2     = 1500  (expected 2000)
constexpr FARE_INNER_C*2 = 2000
sizeof(Zone)=1 sizeof(OldZone)=4
plain enum arithmetic compiles: 7
OInner == 0 ? 1
sizeof(FARE_INNER_C)=4
```

The macro becomes 500 + 500 * 2 = 1500, not 2000, because * is done before +. The constexpr value is evaluated first, so it gives 2000.

3. **enum versus enum class**

A plain enum converts to int silently (OOuter + 5 gives 7). An **enum class** does not: its names stay inside their own scope (Zone::Inner) and never turn into int by themselves. The compiler rejects int x = Zone::Outer;:

Compiler output (g++ -std=c++17 t2_err.cpp):

```
t2_err.cpp:2:28: error: cannot convert 'Zone' to 'int' in initialization
```

So an enum class cannot be mixed with ordinary numbers by mistake. It can also use a small type (1 byte against 4 for a plain enum).

4. **Naming rules and conventions**

- **Rules:** letters, digits and underscore only; cannot start with a digit; case-sensitive; cannot be a keyword.
- **Conventions used:** constants in UPPER_SNAKE_CASE with the unit in the name (MIN_BALANCE_PAISE); types in PascalCase (Zone); functions and variables in camelCase (canTravel); private members end with _ (balance_).

5. **The single set of constants I use**

```
// fare_constants.h : the ONLY place where fares and limits are written
enum class Zone { Inner, Middle, Outer };
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};  // Rs 10, Rs 20, Rs 30
constexpr int  PEAK_SURCHARGE_PERCENT = 20;
constexpr int  CONCESSION_PERCENT     = 50;             // assumption A1
constexpr long long MIN_BALANCE_PAISE = 5000;           // Rs 50 must remain
constexpr int  PEAK_AM_START = 8 * 60,  PEAK_AM_END = 10 * 60;  // [08:00, 10:00)
constexpr int  PEAK_PM_START = 17 * 60, PEAK_PM_END = 20 * 60;  // [17:00, 20:00)
```

To revise a fare, edit one line in this header and recompile.

6. Alternative concept: #define macros and plain integers

What changes with #define: no type (INNER + 5 is accepted), no scope (a macro named INNER replaces that word everywhere), and extra brackets are needed (see 1500 vs 2000 above).

Memory. A macro uses no storage. A constexpr value normally uses none either. Only a constexpr array used at run time (BASE_PAISE, 24 bytes) takes space, which is O(z) for z zones, same as a macro array.

Time. Both are resolved at compile time, so cost is O(1) and identical. The choice affects safety and maintenance, not speed or memory.

TASK 3 – Choose Data Types and Check Type Compatibility

Select a C++ data type for every value in the system.

Discuss:

- The type chosen (bool, char, int, long long, unsigned, float, double, auto) for fares, balances, times, zone numbers and journey counts, with sizeof on your system
- Why money should be stored in integer paise and not as double rupees
- Type compatibility rules, and what the compiler does with int, double and bool values in the same expression
- The ranges from <climits> and whether int is enough for a day's collection across the network

Alternative Concept and Complexity Analysis

If money were stored as double rupees instead of long long paise, what would the solution be? Analyse the time per addition (O(1) in both), the space per value (8 bytes in both), and measure how the rounding error grows when n fares are added.

Ans:

Data Types, Sizes and Compatibility

I wrote t3.cpp to print sizeof and the limits on my machine (g++ 13.3.0, 64-bit Linux) and chose the types from them.

Why money is stored as integer paise

A double stores numbers in binary, and values like 0.1 or 10.8 cannot be stored exactly. My program printed:

Output of t3.cpp (precision part):

```
0.1+0.2 = 0.30000000000000004
10.8 stored as 10.800000000000001
...
ten times 0.10 == 1.00 ? false
double fare 0.1*3 == 0.3 ? false
```

This is the "off by a paisa" defect. An integer such as 1080 paise is always exact, so integer paise is the fix.

Type compatibility and the compiler

When int, double and bool meet in one expression, bool and char become int first; then if one side is double, the other becomes double; and a signed value meeting an unsigned one becomes unsigned. Examples: true + 2 is 3, 7 / 2.0 is 3.5, -1 < 1u is false (see Task 4).

Ranges and the daily collection

The whole-network daily total (Task 10) is 5,626,462,900 paise. INT_MAX is 2,147,483,647, so an int overflows after about 1.14 million journeys, which is less than one day of traffic. A long long holds up to 9.22×10^18, so I use long long for every amount.

Alternative concept: double rupees

Solution with double rupees: balance as double, table {10.0, 20.0, 30.0}, surcharge fare * 1.2, test balance - fare >= 50.0. It looks simpler, but every comparison of money can be hit by rounding error.

Time and space. One addition is O(1) and each value is 8 bytes in both. I timed 100 million additions: long long 31 ms, double 124 ms (the double loop also converts int to double). Same order, slightly larger constant for double.

Rounding error. I added the same fare n times in double and compared with the exact integer-paise total:

Table 7: Absolute error of a double sum against the exact integer-paise sum (measured)

n fares	Exact total (Rs)	Error for Rs 10.80 each (Rs)	Error for Rs 0.10 each (Rs)
10	108.00	−1.421×10^-14	−1.110×10^-16
1,000	10,800.00	−7.822×10^-11	−1.407×10^-12
100,000	1,080,000.00	1.997×10^-6	1.885×10^-8
1,000,000	10,800,000.00	6.423×10^-5	1.333×10^-6
3,000,000	32,400,000.00	1.554×10^-3	1.957×10^-6
10,000,000	108,000,000.00	−1.891×10^-2	−1.610×10^-4

The error grows roughly with n: about 10^-14 Rs at n = 10 and about 0.019 Rs (nearly two paise) at n = 10^7 for Rs 10.80. With long long paise the error is always 0. For today's fares (Rs 10, 20, 30 and their 20% and 50% variants) doubles happen to be exact, but a value like Rs 10.80 (e.g. a 10% discount then 20% surcharge) would break it.

TASK 4 – Control Implicit Conversions

Identify every place in the program where C++ converts a value from one type to another without being told.

Discuss:

- Integer division and integer-to-double conversion
- Comparison of signed and unsigned values, and the compiler warning it produces
- Narrowing conversions, and how brace initialisation (int x{3.7};) prevents them
- static_cast and when an explicit cast is needed

Predict the Output. Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

int main()
{
   int a = 7, b = 2;
   double c = a / b;
   double d = static_cast<double>(a) / b;
   std::cout << c << ' ' << d << '\n';

   char ch = 'A' + 2;
   std::cout << ch << ' ' << ch + 1 << '\n';

   bool flag = 5;
   std::cout << flag << ' ' << std::boolalpha << flag << '\n';

   unsigned int u = 1;
```

```
    int s = -1;
    std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

Alternative Concept and Complexity Analysis

If every conversion in the program were written explicitly with static_cast (and brace initialisation were used everywhere), instead of relying on implicit conversions, what would the solution be? Analyse whether the running time or memory changes, and which errors are caught at compile time instead of at run time.

Ans:

Implicit Conversions

My predicted output (written before running)

```
3 3.5
C 68
1 true
s >= u
```

Actual output

Compiled with g++ -std=c++17 -Wall -Wextra t4.cpp -o t4. The output was exactly the four lines predicted above; the compiler gave one warning:

Compiler warning:

```
t4.cpp:18:21: warning: comparison of integer expressions of different signedness: 'int'
    and 'unsigned int' [-Wsign-compare]
```

Prediction and output were identical. Each line is explained below.

Surprises and the correction

Two surprises: 7 / 2 gives 3 even when stored in a double, and -1 < 1u is false. Both are dangerous in a fare program. Brace initialisation stops narrowing at compile time. I tested int x{3.7}; bool f{5};:

Compiler output (g++ -std=c++17 t4_narrow.cpp):

```
t4_narrow.cpp:1:20: error: narrowing conversion of '3.7000000000000002e+0' from 'double'
    to 'int' [-Wnarrowing]
t4_narrow.cpp:1:33: error: narrowing conversion of '5' from 'int' to 'bool'
    [-Wnarrowing]
```

Correction: write static_cast<double>(a) / b for exact division, and compare signed and unsigned values after casting both to long long. With these, -1 < 1u gives the correct answer (true). static_cast is needed when a conversion is wanted but the compiler would not do it, or would do it unsafely.

Alternative concept: every conversion explicit

Solution with every conversion explicit: use static_cast everywhere and brace-initialise all variables.

Time and memory. static_cast on built-in types produces the same machine code as an implicit conversion. 100 million iterations took 49.7 ms (implicit) and 48.7 ms (explicit), so no real difference. Both are O(1).

Caught earlier. With braces the compiler rejects int x{3.7} and bool f{5} at compile time; with implicit conversion 3.7 is silently cut to 3 and shows up later as a wrong fare.

TASK 5 – Write Correct Expressions with the Right Precedence

Write the expressions for the peak check, the surcharge and the concession.

Discuss:

- Operator precedence and associativity in each expression, including the conditional operator ?: and compound assignment
- The difference between x++ and ++x, and why i = i++ is a mistake
- The result of / and % with negative operands
- Why (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20) is correct and hour >= 8 && hour < 10 || hour >= 17 && hour < 20 is equivalent but harder to read

Predict the Output. Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

int main()
{
   int x = 5, y = 3;
   std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

   int z = x++ + ++y;
   std::cout << z << ' ' << x << ' ' << y << '\n';

   std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
   std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';

   int k = 0;
   for (int i = 0; i < 10; i += 3) k += i;
   std::cout << k << '\n';
}
```

Alternative Concept and Complexity Analysis

If the peak-hour check were done with a lookup table bool peak[24] indexed by the hour, instead of a combined && / || expression, what would the solution be? Analyse the time per check (O(1) in both, but fewer comparisons with the table) and the extra space of 24 bytes.

Ans:

Expressions, Precedence and Associativity

1. **The three expressions of the fare system**

```
// peak check (minutes since midnight, so 8:00 and 17:00 are peak; 10:00 and 20:00 are
   not)
int t = hour * 60 + minute;
bool peak = (t >= 8 * 60 && t < 10 * 60) || (t >= 17 * 60 && t < 20 * 60);

// surcharge, integer paise, rounded half-up
fare += (fare * PEAK_SURCHARGE_PERCENT + 50) / 100;

// concession
if (concession) fare -= (fare * CONCESSION_PERCENT + 50) / 100;
```

Precedence: in hour * 60 + minute, * is done before +. In the peak check, comparisons come first, then &&, then ||. Brackets in (fare * 20 + 50) / 100 force the + before the /. * and / go left to right, so fare * 20 / 100 means (fare * 20) / 100 (multiplying first keeps precision). Compound assignment += is lowest and goes right to left.

2. x++ versus ++x, and why i = i++ is a mistake

x++ gives the old value and then adds 1; ++x adds 1 and then gives the new value. I tested both and the mistaken i = i++:

```
#include <iostream>
int main() { int i = 5; i = i++; std::cout << i << '\n'; int j = 5; int k = j++; int m =
  ++j; std::cout << k << ' ' << m << ' ' << j << '\n'; }
```

Output and compiler warning (g++ -std=c++17 -Wall -Wextra t5_inc.cpp):

```
t5_inc.cpp:2:27: warning: operation on 'i' may be undefined [-Wsequence-point]
5
5 7 7
```

j++ gave 5 (old value) and ++j gave 7. In i = i++ the old value 5 is written back, so the increment is lost. Before C++17 this was undefined behaviour. What was meant is i++;.

3. / and % with negative operands

Since C++11, division truncates toward zero and the remainder takes the sign of the left operand. So 10 / 3 = 3, 10 % 3 = 1, -10 / 3 = -3, -10 % 3 = -1.

4. Brackets and readability

&& is done before ||, so both forms mean the same. I still use brackets because they show the two peak windows clearly, and the compiler (-Wparentheses) suggests them for mixed && and ||.

5. Prediction for the Predict-the-Output program

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Actual output

Compiled with g++ -std=c++17 -Wall -Wextra t5.cpp -o t5, no warnings. The output was exactly the five lines predicted above.

Prediction and output were identical. The operators in each line:

Table 9: Operators involved in every line

Line	Operators and the order they apply	Result
x + y * 2 and (x + y) * 2	* before +: 5 + (3×2) = 11. With brackets + first: (5+3)×2 = 16	11 16
int z = x++ + ++y;	x++ yields 5 and makes x = 6; ++y makes y = 4 and yields 4; then +: z = 5 + 4. The two increments touch different variables, so the result is well defined. Then cout prints z, x, y	9 6 4
(x > y ? x : y)	Brackets, then >: 6 > 4 true, so ?: yields x = 6	6
x % 4 * 2	% and * have the same precedence and associate left to right: (6 % 4) 2 = 2 2	4
10 / 3, 10 % 3	Integer division truncates: 3; remainder 1	3 1
-10 / 3, -10 % 3	Unary minus first, then truncate toward zero: −3; remainder takes dividend's sign: −1	-3 -1
for (i = 0; i < 10; i += 3) k += i;	i takes 0, 3, 6, 9 (+= is compound assignment); k = 0+3+6+9	18

Alternative concept: lookup table

Solution with a lookup table: build a 24-entry table once and index it by hour:

```
bool peakTable[24];
for (int h = 0; h < 24; ++h)
    peakTable[h] = (h >= 8 && h < 10) || (h >= 17 && h < 20);   // filled once at
        start-up
...
bool peak = peakTable[hour];                                    // one array access per
    check
```

This works only because both windows start and end on whole hours. A window starting at 17:30 would need a minute-level test.

Time. The expression needs at most 4 comparisons and 2 logical operators; the table needs one lookup. Both are O(1). Timed over 50 million hours: expression 78.4 ms, table 75.0 ms.

Space. The table adds 24 bytes (O(1)); the expression adds none. The table also stores the peak rule a second time, so it must be updated whenever the rule changes.

TASK 6 – Design the Control Structures

Design the decision and looping logic for computing a fare and processing a day's journeys.

Discuss:

- When to use if–else, a switch on the zone, and the conditional operator
- How a for loop, a range-based for loop and a while loop could process the journeys, and which is best
- Where break and continue are needed (for example, skipping an invalid journey)
- What happens when no case of a switch matches
- Write pseudocode for the fare computation following the SIMATS Engineering – Standard for Algorithmic Pseudocode before writing C++ code.

Complete the Code. Complete TODO 1 and TODO 2 below using a switch statement and integer paise, and execute the program.

```
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, Rs 20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000; // Rs 50 must remain on the card

long long fare(Zone z, bool peak)
{
    // TODO 1: Use a switch on z to pick the base fare from BASE_PAISE, then add
    // PEAK_SURCHARGE_PERCENT % in peak hours using integer arithmetic only
}

bool canTravel(long long balance, long long f)
{
    // TODO 2: Return true only if the balance left after paying the fare is at least
    // MIN_BALANCE_PAISE
}
```

```
int main()
{ ... }   // unchanged; given in the assignment
```

Expected output:

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

Add a journey at exactly 10 AM and at exactly 8 PM and check that your peak rule treats them correctly.

Alternative Concept and Complexity Analysis

If the base fare were looked up from an array indexed by the zone (constexpr long long FARE[] = {1000, 2000, 3000}) instead of a switch, what would the solution be? Analyse the time per lookup (O(1) in both; the switch may become a jump table), the space of the array, and how each design copes with a fourth zone.

Ans:

Control Structures and the Fare Computation

1. **Choosing the control structure**

Table 10: Which structure for which decision

Decision	Structure chosen	Reason
Base fare from the zone	switch on Zone	One variable compared with a few discrete values; the compiler warns (-Wswitch) if an enumerator is not handled
Peak or off-peak; concession or not	if / else	Conditions are ranges or combined tests, which a switch cannot express
Choosing between two values inside an expression (e.g. printing "Denied")	Conditional operator ?:	Short two-way choice returning a value; clear only when it fits on one line
Allow or deny the journey	if / else on canTravel()	Two outcomes with different statements

Table 11: Looping over the day's journeys

Loop	Sketch	Judgement
for	for (size_t i = 0; i < n; ++i) { ... trips[i] ... }	Works, but the index can run past the end if n is wrong
range-based for	for (const Trip& t : trips) { ... }	Best: visits every journey exactly once, no index, no copy (const reference)
while	while (std::cin >> zone >> h >> m) { ... }	Best when the number of journeys is unknown and read from input (Task 8)

break and continue. break ends a switch case (without it, execution falls into the next case) or stops the loop. continue skips an invalid journey and moves to the next, so one bad record does not stop the day. If no case matches, a switch does nothing; since base starts at 0, an unhandled zone would give a free ride (see Task 12).

***2.* Pseudocode**

The SIMATS pseudocode standard was not supplied, so I used the usual form (named algorithm, INPUT/OUTPUT, BEGIN/END, indentation, <- for assignment). Re-check it against the standard before submission.

```
ALGORITHM ComputeFare
INPUT  : z (zone: Inner | Middle | Outer), peak (boolean)
OUTPUT : fare in paise (integer)
CONSTANTS: BASE_PAISE = [1000, 2000, 3000], PEAK_SURCHARGE_PERCENT = 20
BEGIN
   SWITCH z
      CASE Inner  : base <- BASE_PAISE[0]
      CASE Middle : base <- BASE_PAISE[1]
      CASE Outer  : base <- BASE_PAISE[2]
   END SWITCH
   IF peak THEN
      base <- base + base * PEAK_SURCHARGE_PERCENT / 100      // integer arithmetic
         only
   END IF
   RETURN base
END

ALGORITHM CanTravel
INPUT  : balance, fare (paise)
OUTPUT : TRUE or FALSE
BEGIN
   RETURN (balance - fare) >= MIN_BALANCE_PAISE
END

ALGORITHM ProcessDay
INPUT  : list of journeys, starting balance
BEGIN
   FOR EACH journey IN list
      IF journey is invalid THEN CONTINUE
      f <- ComputeFare(journey.zone, journey.peak)
      IF CanTravel(balance, f) THEN balance <- balance - f
      ELSE report "Denied"
   END FOR
END
```

***3.* Completed code (TODO 1 and TODO 2 replaced, rest unchanged)**

```
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, Rs 20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000; // Rs 50 must remain on the card

long long fare(Zone z, bool peak)
{
   long long base = 0;
   switch (z) {                          // TODO 1
```

```
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
    }
    if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;  // integer arithmetic only
    return base;
}

bool canTravel(long long balance, long long f)
{
    return balance - f >= MIN_BALANCE_PAISE;        // TODO 2
}

int main()
{
    long long balance = 10000; // Rs 100.00 in paise
    struct Trip { Zone zone; bool peak; };
    Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
                      << std::setfill('0') << f % 100 << std::setfill(' ');
        } else {
            std::cout << "Denied (low balance)";
        }
        std::cout << " balance Rs " << balance / 100 << '\n';
    }
}
```

base += base * PEAK_SURCHARGE_PERCENT / 100 multiplies first (3000 × 20 = 60000) and then divides (600), so the surcharge is exact in paise. Compiled with -Wall -Wextra, no warnings.

Trace: Inner off-peak costs 1000, balance 10000 → 9000 (allowed). Outer peak costs 3000 + 600 = 3600, balance 9000 → 5400 (allowed). Middle off-peak costs 2000, which would leave 3400 < 5000, so it is denied and the balance stays 5400. (The program prints balance / 100, which drops the paise; Task 8 prints both parts.)

***4.* Journeys at exactly 10 AM and 8 PM**

The given fare(Zone, bool) receives the peak flag from outside, so the boundary rule lives in the function that makes the flag. I wrote isPeak(hour, minute) in core.h and tested the boundaries:

```
inline bool isPeak(int hour, int minute)
{
    int t = hour * 60 + minute;                         // minutes since midnight
    return (t >= PEAK_AM_START && t < PEAK_AM_END) || (t >= PEAK_PM_START && t <
        PEAK_PM_END);
}
```

Output of t6b.cpp:

```
7:59 -> off-peak
8:00 -> peak
9:59 -> peak
10:00 -> off-peak
16:59 -> off-peak
17:00 -> peak
19:59 -> peak
20:00 -> off-peak
```

10:00 AM and 8:00 PM are off-peak; 8:00 and 17:00 are peak. The old 10 AM / 8 PM inconsistency is gone because both ends use < on the same minute scale.

Alternative concept: array lookup

What the solution would be.

```
constexpr long long FARE[] = {1000, 2000, 3000};
long long fare(Zone z, bool peak)
{
   long long base = FARE[static_cast<int>(z)];       // no switch
   if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
   return base;
}
```

Time. Both are O(1). Timed over 50 million zones: switch 57.7 ms, array 73.0 ms; the difference depends on compiler output, so neither is really slower. **Space.** The array uses 3 × 8 = 24 bytes (O(z)); the switch keeps its values in instructions.

Fourth zone. With a switch, I add an enum value and a case; if I forget the case, the compiler only warns and the fare becomes 0. With an array, I add one element; if I forget, FARE[3] reads outside the array with no warning. The array is shorter and scales better but needs a static_assert on its size, which is why Task 12 uses it.

TASK 7 – From Procedural to Object-Oriented

Rewrite the fare logic in two styles.

Discuss:

- A procedural version with separate variables and free functions
- An object-oriented version with a SmartCard class that keeps its balance private and offers deduct() and canTravel()
- How the class prevents a negative balance
- Which version is easier to change when the minimum balance changes

Alternative Concept and Complexity Analysis

If the balance were kept in a public data member instead of a private member with deduct() and canTravel(), what would the solution be? Analyse whether time or space changes (they do not), and explain which rule of the system can then be broken by any part of the program.

Ans:

Procedural versus Object-Oriented Fare Logic

1. **Procedural version**

```
// Procedural version: global variables + free functions
#include <iostream>
long long balance = 10000; // global: any function can change it
```

```
const long long BASE[] = {1000, 2000, 3000};
const long long MIN_BAL = 5000;
long long fareOf(int zone, bool peak)
{
   long long f = BASE[zone];
   if (peak) f += f * 20 / 100;
   return f;
}
bool canTravel(long long f) { return balance - f >= MIN_BAL; }
void deduct(long long f)
{
   balance -= f;
} // nothing stops a caller skipping canTravel()
int main()
{
   int zones[] = {0, 2, 1};
   bool peaks[] = {false, true, false};
   for (int i = 0; i < 3; ++i)
   {
      long long f = fareOf(zones[i], peaks[i]);
      if (canTravel(f))
      {
         deduct(f);
         std::cout << "paid " << f << " balance " << balance << '\n';
      }
      else
         std::cout << "denied balance " << balance << '\n';
   }
   deduct(9999); // legal call -> negative balance
   std::cout << "after careless deduct: " << balance << '\n';
}
```

Output (g++ -std=c++17 -Wall -Wextra, no warnings):

```
paid 1000 balance 9000
paid 3600 balance 5400
denied balance 5400
after careless deduct: -4599
```

The first three lines are correct. The last shows the weakness: deduct(9999) is a legal call and nothing stops it, so the balance becomes −4599. The old negative-balance bug is reproduced.

2. **Object-oriented version**

```
// Object-oriented version: SmartCard keeps balance private
#include <iostream>
class FareTable
{
   static constexpr long long BASE[3] = {1000, 2000, 3000};

 public:
   static long long fare(int zone, bool peak)
   {
      long long f = BASE[zone];
      if (peak) f += f * 20 / 100;
```

```
        return f;
    }
};
class SmartCard
{
    long long balance_; // private
    static constexpr long long MIN_BAL = 5000;

  public:
    explicit SmartCard(long long b) : balance_(b) {}
    long long balance() const { return balance_; }
    bool canTravel(long long f) const { return balance_ - f >= MIN_BAL; }
    bool deduct(long long f)
    {
        if (!canTravel(f)) return false;
        balance_ -= f;
        return true;
    }
};
int main()
{
    SmartCard card(10000);
    int zones[] = {0, 2, 1};
    bool peaks[] = {false, true, false};
    for (int i = 0; i < 3; ++i)
    {
        long long f = FareTable::fare(zones[i], peaks[i]);
        if (card.deduct(f))
            std::cout << "paid " << f << " balance " << card.balance() << '\n';
        else
            std::cout << "denied balance " << card.balance() << '\n';
    }
    std::cout << "careless deduct accepted? " << std::boolalpha << card.deduct(9999)
              << ", balance " << card.balance() << '\n';
}
```

Output (g++ -std=c++17 -Wall -Wextra, no warnings):

```
paid 1000 balance 9000
paid 3600 balance 5400
denied balance 5400
careless deduct accepted? false, balance 5400
```

The first three lines match the procedural version. The careless deduct(9999) is refused and the balance stays 5400.

3. **How the class prevents a negative balance**

balance_ is private, so it can change only through deduct() or recharge() (positive amounts only). deduct() checks canTravel() itself and changes nothing when balance_ - f < 5000. So the balance can never go below zero. The constructor sets this rule and every public function keeps it.

Table 12: Comparison of the two versions

Point	Procedural	Object-oriented
Where the balance lives	Global variable, visible to all	Private member of SmartCard

Point	Procedural	Object-oriented
Who enforces Rs 50	Every caller must call canTravel() first	deduct() enforces it itself
Result of careless deduct(9999)	balance −4599 (rule broken)	refused; balance 5400
Change the minimum balance	One constant, but every function using balance must be reviewed	One constant inside SmartCard; only 4 member functions can use it
Amount of code	Slightly shorter	A few more lines (class, constructor)
Time and space per journey	O(1), 8 bytes	O(1), 8 bytes (measured)

Easier to change: the object-oriented version. In both, 5000 appears once, but in the procedural program any new function can call deduct() directly and skip the rule. In the OO program the rule is inside the class and the compiler stops other code from touching the balance.

4. **Alternative concept: public balance**

```
// Alternative: balance as a PUBLIC data member
#include <iostream>
struct SmartCard { long long balance; };
int main()
{
   SmartCard card{10000};
   card.balance -= 9999;      // compiles; nothing checks the Rs 50 rule
   card.balance = -500;       // compiles; negative balance
   std::cout << card.balance << '\n';
}
```

Output (compiles with no warning):

```
-500
```

Solution with a public balance: struct SmartCard { long long balance; };. Both card.balance -= 9999; and card.balance = -500; compile and run. Time and space do not change (O(1), 8 bytes), but any code can now break R1 (Rs 50 minimum), the non-negative balance, and the positive-recharge rule.

TASK 8 – Read and Print Data Safely

Design the input and output of the gate program.

Discuss:

- Reading values with cin and checking cin.fail() when the operator types letters instead of numbers
- Clearing the error state with cin.clear() and discarding the bad input
- Formatting money with <iomanip> (setw, setfill, fixed, setprecision)
- Printing a neat journey report

Alternative Concept and Complexity Analysis

If each input line were read with std::getline() and converted with std::stoi() inside try/catch, instead of reading numbers directly with cin >>, what would the solution be? Analyse the time to process a line of length L (O(L) in both) and the buffer space needed, and compare how each recovers from invalid input.

Ans:

Safe Input and Formatted Output

1. **What goes wrong with unchecked cin**

If the operator types abc where a number is expected, cin >> n goes into a fail state, stores 0 and leaves the letters in the buffer. Every later read then does nothing. Recovery has three steps: check if (!(cin >> n)), call cin.clear(), then cin.ignore(max, '\n') to throw away the bad line.

2. **The gate program**

```
#include "core.h"
#include <iomanip>
#include <iostream>
#include <limits>
int main()
{
   long long balance = 10000;
   int n;
   std::cout << "Journeys: ";
   while (!(std::cin >> n) || n < 0)
   {
      std::cout << "Invalid count, try again\n";
      std::cin.clear();
      std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
      if (std::cin.eof()) return 1;
   }
   std::cout << '\n'
             << std::left << std::setw(4) << "No" << std::setw(6) << "Zone"
             << std::setw(7) << "Time" << std::right << std::setw(10) << "Fare(Rs)"
             << std::setw(13) << "Balance(Rs)" << '\n';
   for (int i = 1; i <= n; ++i)
   {
      int zone, h, m;
      if (!(std::cin >> zone >> h >> m) || zone < 0 || zone > 2 || h < 0 || h > 23 ||
         m < 0 || m > 59)
      {
         std::cout << "Journey " << i << ": invalid input, skipped\n";
         std::cin.clear();
         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
         continue;
      }
      long long f = fare(static_cast<Zone>(zone), isPeak(h, m), false);
      std::cout << std::left << std::setw(4) << i << std::setw(6) << zone
                << std::setfill('0') << std::right << std::setw(2) << h << ':'
                << std::setw(2) << m << std::setfill(' ') << "  ";
      if (canTravel(balance, f))
      {
         balance -= f;
         std::cout << std::setw(10) << std::fixed << std::setprecision(2)
                   << f / 100.0 << std::setw(13) << balance / 100.0 << '\n';
      }
      else
         std::cout << std::setw(10) << "DENIED" << std::setw(13) << balance / 100.0
                   << '\n';
   }
```

```
}
```

isPeak(), fare() and canTravel() come from core.h. Each journey is read as zone hour minute (zone 0–2). Bad values are rejected like letters. setw sets width, setfill('0') pads the time (09:30), and fixed with setprecision(2) prints two decimals. Money is divided by 100.0 only when printing; all arithmetic stays in paise.

***3.* Test with invalid input**

Input file t8.in contains a journey with letters in the hour (2 abc 10) and one with an unknown zone (3 10 0):

t8.in:

```
5
0 9 30
2 abc 10
1 18 5
3 10 0
2 20 0
```

Output of ./t8 < t8.in:

```
Journeys:
No  Zone  Time     Fare(Rs)  Balance(Rs)
1   0     09:30       12.00        88.00
Journey 2: invalid input, skipped
3   1     18:05       24.00        64.00
Journey 4: invalid input, skipped
5   2     20:00      DENIED        64.00
```

Journeys 2 and 4 are reported and skipped, and journey 5 is still processed. Journey 5 (Outer at 20:00) is off-peak, Rs 30.00, but the balance would drop to Rs 34.00 (< Rs 50), so it is denied. Journey 3 (18:05) is peak: Rs 20 + 20% = Rs 24.00. For a bad count (x, -3, 2 ..., file t8b.in):

Output of ./t8 < t8b.in:

```
Journeys: Invalid count, try again
Invalid count, try again

No  Zone  Time     Fare(Rs)  Balance(Rs)
1   0     08:00       12.00        88.00
2   2     10:00       30.00        58.00
```

The letters and the negative count are rejected and the question repeats; then 2 journeys run. 08:00 is peak (12.00) and 10:00 is off-peak (30.00).

Alternative concept: getline() and stoi() with try/catch

```
#include <iostream>
#include <string>
int main()
{
   std::string line;
   while (std::getline(std::cin, line)) {
      try {
         size_t pos; int v = std::stoi(line, &pos);
         if (pos != line.size()) throw std::invalid_argument("trailing");
         std::cout << "ok " << v << '\n';
      } catch (const std::exception&) { std::cout << "bad line: '" << line << "'\n"; }
```

```
    }
}
```

Output with input t8g.in (12, 7x, abc, " 5", 99999999999):

```
ok 12
bad line: '7x'
bad line: 'abc'
ok 5
bad line: '99999999999'
```

Solution with getline and stoi: read each line into a string and convert with stoi; exceptions report errors. abc throws invalid_argument, 99999999999 throws out_of_range, and 7x is accepted as 7, so I also check pos != line.size().

Time. Both examine each character once: O(L). Timed over 1,000,000 lines: >> 19.8 ms, getline + stoi 20.5 ms. **Space.** cin >> needs no per-line storage; getline needs a string of L bytes (O(L)).

Recovery. With cin >> the stream stays failed until clear() and ignore() are called. With getline each line is independent, so a bad line only throws for that line. This is safer for line-based records, but I used cin >> in the main program because it is simpler for numbers.

TASK 9 – Implement and Test the Fare Logic **4 Marks**

Implement the complete fare logic and test it.

Discuss:

- Normal journeys in each zone, peak and off-peak
- Boundary cases: exactly 8:00, 10:00, 17:00 and 20:00; a balance that leaves exactly Rs 50
- Invalid cases: an unknown zone, a negative recharge, letters typed for the hour
- A small set of 5 journeys whose total you calculate by hand

Alternative Concept and Complexity Analysis

If each journey were tested against a table of test cases (inputs and expected fares) in a loop, instead of separate hand-written checks, what would the solution be? Analyse the time for t tests (O(t)) and the space of the table, and how easily a new boundary case can be added.

Implementation and Testing of the Fare Logic

The fare logic is in core.h (Tasks 2, 5, 6). The test program t9.cpp uses a table of cases; each row has an input and the fare I worked out by hand first:

```
#include "core.h"
#include <cstdlib>
#include <iostream>
#include <sstream>
#include <string>

struct Case
{
    const char *name;
    int zone, h, m;
    bool conc;
    long long expected;
};
// Parses a zone/hour text the way the gate does; returns false for invalid input
bool parseHour(const std::string &s, int &out)
{
```

```
    std::istringstream in(s);
    in >> out;
    return !in.fail() && in.eof() && out >= 0 && out <= 23;
}
int main()
{
    Case cases[] = {
        {"Inner off-peak", 0, 12, 0, false, 1000},
        {"Middle off-peak", 1, 12, 0, false, 2000},
        {"Outer off-peak", 2, 12, 0, false, 3000},
        {"Inner peak AM", 0, 9, 0, false, 1200},
        {"Middle peak PM", 1, 18, 0, false, 2400},
        {"Outer peak AM", 2, 8, 30, false, 3600},
        {"Boundary 08:00", 0, 8, 0, false, 1200},
        {"Boundary 09:59", 0, 9, 59, false, 1200},
        {"Boundary 10:00", 0, 10, 0, false, 1000},
        {"Boundary 17:00", 0, 17, 0, false, 1200},
        {"Boundary 19:59", 0, 19, 59, false, 1200},
        {"Boundary 20:00", 0, 20, 0, false, 1000},
        {"Concession off", 2, 12, 0, true, 1500},
        {"Concession peak", 2, 9, 0, true, 1800},
    };
    int pass = 0, total = 0;
    std::cout << "No  Case              Expected  Actual  Result\n";
    for (const Case &c : cases)
    {
        long long a = fare(static_cast<Zone>(c.zone), isPeak(c.h, c.m), c.conc);
        bool ok = (a == c.expected);
        ++total;
        pass += ok;
        std::cout << (total < 10 ? " " : "") << total << "  " << c.name
                  << std::string(17 - std::string(c.name).size(), ' ') << c.expected
                  << "      " << a << "    " << (ok ? "PASS" : "FAIL") << '\n';
    }
    // balance leaving exactly Rs 50
    bool b1 = canTravel(6000, 1000), b2 = canTravel(5999, 1000),
         b3 = canTravel(6000, 1001);
    std::cout << "Balance 6000, fare 1000 (leaves exactly 5000): "
              << (b1 ? "allowed" : "denied") << "  expected allowed "
              << (b1 ? "PASS" : "FAIL") << '\n';
    std::cout << "Balance 5999, fare 1000 (leaves 4999):          "
              << (b2 ? "allowed" : "denied") << "  expected denied  "
              << (!b2 ? "PASS" : "FAIL") << '\n';
    std::cout << "Balance 6000, fare 1001 (leaves 4999):          "
              << (b3 ? "allowed" : "denied") << "  expected denied  "
              << (!b3 ? "PASS" : "FAIL") << '\n';
    // invalid
    int zone = 7;
    bool badZone = !(zone >= 0 && zone <= 2);
    long long recharge = -200;
    bool badRecharge = recharge <= 0;
    int h;
    bool badHour = !parseHour("ab", h);
```

```
    std::cout << "Unknown zone 7 rejected:      " << (badZone ? "PASS" : "FAIL") << '\n'
              << "Negative recharge rejected:  " << (badRecharge ? "PASS" : "FAIL")
              << '\n'
              << "Hour 'ab' rejected:          " << (badHour ? "PASS" : "FAIL") << '\n'
              << "Hour '25' rejected:          "
              << (!parseHour("25", h) ? "PASS" : "FAIL") << '\n';
    // 5 journeys by hand: Inner 12:00=1000, Middle 09:00=2400, Outer 18:30=3600, Inner
    // 10:00=1000, Outer 20:00=3000 -> 11000
    struct J
    {
       int z, h, m;
    } js[] = {{0, 12, 0}, {1, 9, 0}, {2, 18, 30}, {0, 10, 0}, {2, 20, 0}};
    long long total5 = 0;
    for (auto j : js) total5 += fare(static_cast<Zone>(j.z), isPeak(j.h, j.m), false);
    std::cout << "Five-journey total: hand 11000, program " << total5 << "  "
              << (total5 == 11000 ? "PASS" : "FAIL") << '\n';
    std::cout << pass << "/" << total << " table cases passed\n";
}
```

Compiled with g++ -std=c++17 -Wall -Wextra t9.cpp -o t9, no warnings.

Hand calculation of the five journeys: Inner 12:00 = 1000; Middle 09:00 peak = 2000 + 400 = 2400; Outer 18:30 peak = 3000 + 600 = 3600; Inner 10:00 = 1000; Outer 20:00 = 3000. Total = 11,000 paise (Rs 110.00), matched by the program.

Alternative concept: table-driven tests

Solution: t9.cpp already uses a Case cases[] array and one for loop that compares computed and expected fares. The alternative is a separate hand-written if for each case.

Time and space. For t tests: O(t) time and O(t) space in both. For 1,000,000 cases the checks took 1.04 ms and the table 16 MB. A new boundary case is one new row of data in the table, but a new statement with hand-written checks.

TASK 10 – Compare Expectations with Execution **4 Marks**

Compare what you expected with what actually happened.

Discuss:

- Whether the double and integer-paise totals for 20,000, 500,000 and 3,000,000 journeys agree, and by how much they differ
- Whether int overflows for the network total
- The compiler warnings you received with -Wall -Wextra and what each meant
- Reasons for any difference

Alternative Concept and Complexity Analysis

If the network total were accumulated in an int, a long long or a double, what would happen at 3,000,000 journeys? Analyse when each type fails (overflow versus loss of precision), and compare the time per addition and the space per variable.

Ans:

Expected versus Observed Behaviour

1. **Experiment**

t10.cpp makes journeys with a fixed-seed generator (zone, hour, and a concession for one journey in five) and adds the fares four ways: long long paise (exact), double rupees, float rupees, and int (the

exact total converted to int). I expected that long long and double would agree, float would lose accuracy at large totals, and int would overflow at network size.

Table 14: Totals compared (paise unless stated)

Journeys	long long (exact)	double rupees × 100	Difference	float (Rs)	int (wrapped)
20,000	37,563,600	37,563,600	0	exact	37,563,600 (fits)
500,000	937,370,000	937,370,000	0	exact	937,370,000 (fits)
3,000,000	5,626,462,900	5,626,462,900	0	−48,977 Rs	1,331,495,604 (overflow)

2. Does int overflow?

Yes, at network size. The exact total 5,626,462,900 paise is more than INT_MAX (2,147,483,647). The wrapped value 1,331,495,604 is the total minus 2^32. (Signed overflow is really undefined behaviour; I converted the exact total to show the usual result.) With an average fare of 1,875 paise, int fails after about 1.14 million journeys.

3. Reasons for the differences

- **double agreed exactly.** Every fare here (Rs 10, 20, 30, × 0.5, × 1.2) is exactly representable and all sums stay below 2^53, so no rounding happened. This is luck of the numbers: Task 3 shows drift for Rs 10.80 fares.
- **float lost Rs 48,977.** A float has about 7 decimal digits. Once the total passes 16,777,216 (2^24), adding Rs 12–36 is rounded away. At 20,000 and 500,000 journeys there was no error.
- **int wrapped.** The type is too small (overflow); the cure is a wider type.

4. Compiler warnings received with -Wall -Wextra

Table 15: Warnings I actually received while building this assignment

Alternative concept: the total in int, long long or double

Table 16: What happens at 3,000,000 journeys (exact total 5,626,462,900 paise)

Accumulator	Result	Failure mode	sizeof	Time per 10^8 additions (measured)
int (paise)	1,331,495,604 (wrapped; undefined behaviour in C++)	Overflow after about 1.14 M journeys	4 bytes	33 ms (unsigned)
long long (paise)	5,626,462,900 – exact	None until 9.2×10^18 (about 4.9×10^15 journeys)	8 bytes	31 ms
double (rupees)	Rs 56,264,629.00 – correct for these fares	Loss of precision: fractional fares drift (Task 3); exact integers only up to 2^53	8 bytes	124 ms
float (rupees)	Rs 56,215,652 – wrong by Rs 48,977	Loss of precision above 2^24 (16.7 million)	4 bytes	not timed

Overflow means the value does not fit (a wrong answer that looks precise). **Loss of precision** means the value fits but is not exact. All additions are O(1) with 4 or 8 bytes per variable. long long is as fast as int and exact, so it is the right choice.

TASK 11 – Recommend and Defend the Final Design **4 Marks**

Recommend the final design for the metro.

Discuss:

- The types, constants and classes you recommend and why
- How a fare revision will be carried out
- The limitations of your design
- Whether the design remains correct for the whole network
- Your recommendation must be based on evidence from your own experiments.

Alternative Concept and Complexity Analysis

If the fare table were read from a configuration file when the gate starts, instead of being compiled into the program, what would the solution be? Analyse the extra start-up time to read z fare values and the space they occupy, against the cost of recompiling and redeploying after every revision.

Final Design Recommendation

1. **Recommended design**

Table 17: Recommended types, constants and classes, with the evidence for each

Element	Recommendation	Evidence from my experiments
Money	long long paise everywhere; rupees only when printing	Task 3: double drifted by up to 0.019 Rs for n = 10^7; Task 10: int wrapped at 3 M journeys; long long exact in every run
Constants	constexpr in one header (fares, percentages, minimum balance, peak limits)	Task 2: macro 500+500 gave 1500 instead of 2000; constexpr gave 2000
Zones	enum class Zone	Task 2: plain enum allowed OOuter + 5; enum class refused it
Peak rule	isPeak() on minutes since midnight with half-open intervals	Task 5/6/9: 10:00 and 20:00 off-peak, 08:00 and 17:00 peak; mutation test failed as expected
SmartCard class	Private balance, canTravel(), deduct() with the check inside	Task 7: careless deduct(9999) made the procedural balance −4599 and was refused by the class; same speed (226 ms vs 234 ms per 50 M)
Fare function	fare() with integer percentage and half-up rounding	Task 6/9: 14/14 table cases and the 5-journey total matched
Input	cin with fail()/clear()/ignore(), or getline + validation for records	Task 8: bad input skipped, processing continued
Compiler flags	-std=c++17 -Wall -Wextra, warnings treated as errors	Task 10: sign-compare, sequence-point, unused-variable and -Wswitch warnings each pointed at a real defect

2. **How a fare revision is carried out**

1. Edit the one line in the constants header (e.g. {1000, 2000, 3000} → {1100, 2200, 3300}).
2. Rebuild with -Wall -Wextra; no other file contains a fare.
3. Update the expected column in the table-driven test (t9) and run it; boundary tests already exist.
4. Deploy the new executable to the gates.

3. **Limitations of the design**

- Assumption A1 (50% concession) and the rounding rule are mine and must be confirmed.

- A fare change needs a rebuild and redeployment (see the alternative below).
- The Task 6 program prints only whole rupees; a real display should show both parts.
- Two gates updating one card at the same time is not handled (a real system needs a database transaction).
- Only three zones, no weekend rule and no pass – Task 12 extends this.

***4.* Is the design correct for the whole network?**

Yes. The 3,000,000-journey total (5,626,462,900 paise) fits easily in a long long. Each journey is O(1) (about 4.5 ns), so 3 million journeys take about 14 ms. Memory is 8 bytes per card, so the design is correct and fast for the whole network.

Alternative concept: fare table read from a configuration file

```
// fares.cfg : one fare in paise per line (Inner, Middle, Outer, ...)
//   1000
//   2000
//   3000
std::vector<long long> loadFares(const std::string& path)
{
   std::ifstream in(path);
   std::vector<long long> v; long long x;
   while (in >> x) v.push_back(x);
   if (v.empty()) throw std::runtime_error("no fares");  // must validate: file may be
      missing/wrong
   return v;                                    // the gate then uses v[zone]
      instead of the constexpr array
}
```

Time. Reading z values is O(z) once at start-up (about 1.2 µs for a 4-line file); the cost per journey is unchanged, O(1). **Space.** z × 8 bytes (32 bytes for 4 zones) in a std::vector.

Recompiling versus a file. Recompiling and redeploying to hundreds of gates takes far longer than 1.2 µs of start-up, so a config file wins when fares change often. It loses on safety: no compile-time checks, a wrong or missing file must be caught at run time, and the file can be edited by mistake.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design	**8 Marks**

Changed Requirement

The metro adds a fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two chosen zones.

Re-evaluate your original design. You may RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis – 2 Marks

Identify which constants, enum values, expressions, switch statements and classes are affected. Evidence Expected: a clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether the switch-based fare logic and the SmartCard class can absorb the changes, using worked examples. Evidence Expected: a technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe and implement the revised design (for example: a fare table indexed by zone, a separate discount rule, a Pass class). Evidence Expected: an appropriate design response with working code.

D. Evidence-Based Defence – 2 Marks

Defend your decision using test results and the trade-offs involved. Evidence Expected: a well-supported final decision integrating tests and the changed requirements.

Re-evaluation of the Design for the Changed Requirements

Assumptions for the new rules (the statement leaves them open, so I fix them and keep each in one place):

- **A6 – Weekend:** Saturday and Sunday. The 10% discount replaces the peak surcharge (no surcharge on weekends, even in peak hours). If the authority meant otherwise, it is a one-line change in fare().
- **A7 – Order:** concession first (50%), then either the weekend discount or the peak surcharge, rounded half-up to the paisa.
- **A8 – Pass:** valid for one month and two chosen zones. A covered journey costs 0, so the Rs 50 minimum is not applied. The pass price is outside this assignment.
- **A9 – Airport:** the fourth and farthest zone, base Rs 60; same surcharge, concession and discount rules.

A. **Impact Analysis**

Table 18: What the changed requirements touch in the original design

Item in the original design	Affected?	How
Constant BASE_PAISE[]	Yes	A fourth element 6000 (Airport) must be added
Constant PEAK_SURCHARGE_PERCENT, CONCESSION_PERCENT, MIN_BALANCE_PAISE	No	Values unchanged
New constant WEEKEND_DISCOUNT_PERCENT = 10	New	A new rule needs a new named constant
enum class Zone { Inner, Middle, Outer }	Yes	New enumerator Airport (and a Count for sizing the table)
switch in fare()	Yes	No case for Airport; compiler warns but the code compiles; a missing case silently gives fare 0
fare(Zone, bool peak) signature	Yes	Only a zone and a peak flag go in; the weekday, the pass and the concession cannot be expressed
isPeak(hour, minute)	No	Unchanged, but must not be applied on a weekend (A6)
Expression for surcharge	Yes	Must now be conditional on not-weekend
Expression for concession	No	Unchanged (applied first)
SmartCard class: balance_, canTravel(), deduct()	Partly	Balance logic unchanged; needs to know about a pass (a free journey must not be refused for low balance)
Journey data	Yes	Needs weekday, concession flag, year and month
Pass	New	A new concept: two zones, a month, a coverage test

Item in the original design	Affected?	How
Input routine (Task 8)	Yes	Zone range 0–3; read the day and date; validate them

B. Evaluation of the Original Design

I tested whether the switch logic and SmartCard can absorb the three changes, using worked examples.

Example 1 – fourth zone. I added only Airport to the original Task 6 code (t12_old.cpp) and asked for an off-peak Airport fare:

Compiler output and program output:

```
t12_old.cpp:8:12: warning: enumeration value 'Airport' not handled in switch [-Wswitch]
Airport fare: 0 paise
```

It compiles with only a warning, and the Airport journey is free (Rs 0.00) instead of Rs 60.00 – a silent revenue loss. The switch works only if every switch is updated by hand.

Example 2 – weekend discount. Airport, Saturday 09:00, student. Expected: Rs 60 → concession Rs 30 → 10% off = Rs 27.00 (no surcharge). The original fare(Zone z, bool peak) cannot receive the day, and with peak = true it adds 20% (Rs 36.00), which the new rule forbids.

Example 3 – pass. A card has an Inner + Middle pass and Rs 10.00. A Middle journey must be free, but deduct(2000) tests 1000 − 2000 ≥ 5000, fails, and refuses it. The class has no place for the pass.

Table 19: Can the original design absorb the change?

C. Adaptation: the revised design

I **modified** the design: I kept what was correct and tested (integer paise, constants in one place, the Rs 50 rule, pct() rounding, SmartCard encapsulation) and changed only what the new rules touch:

- A fare table indexed by zone (BASE_PAISE[ZONE_COUNT], with Zone::Count last) replaces the switch; a new zone is one enumerator and one number.
- One function fare(const Journey&) applies concession, then weekend discount or peak surcharge.
- A Journey struct carries zone, time, weekday, concession and date.
- A Pass class holds two zones and a month; SmartCard::travel() asks the pass first and deducts only if it does not cover the journey.

Figure 2: Revised class design for Task 12 (shaded boxes are new or changed)

```
// Task 12: revised design - 4 zones, weekend discount, monthly pass
#include <iomanip>
#include <iostream>
#include <string>

enum class Zone
{
   Inner,
   Middle,
   Outer,
   Airport,
   Count
};
constexpr int ZONE_COUNT = static_cast<int>(Zone::Count);
```

```
constexpr long long BASE_PAISE[ZONE_COUNT] = {1000, 2000, 3000,
                                    6000}; // Rs 10, 20, 30, 60
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;
constexpr int CONCESSION_PERCENT = 50;
constexpr long long MIN_BALANCE_PAISE = 5000;

struct Journey
{
   Zone farthest;
   int hour, minute;
   int weekday; // weekday 0=Mon..6=Sun
   bool concession;
   int year, month;
};

constexpr bool isWeekend(int weekday) { return weekday >= 5; }
constexpr bool isPeak(int h, int m)
{
   int t = h * 60 + m;
   return (t >= 480 && t < 600) || (t >= 1020 && t < 1200);
}
constexpr long long pct(long long a, int p) { return (a * p + 50) / 100; }

// Fare rule kept in ONE place: weekend discount replaces the peak surcharge (assumption
// A1)
long long fare(const Journey &j)
{
   long long f = BASE_PAISE[static_cast<int>(j.farthest)];
   if (j.concession) f -= pct(f, CONCESSION_PERCENT);
   if (isWeekend(j.weekday))
      f -= pct(f, WEEKEND_DISCOUNT_PERCENT);
   else if (isPeak(j.hour, j.minute))
      f += pct(f, PEAK_SURCHARGE_PERCENT);
   return f;
}

class Pass
{ // monthly pass for two chosen zones
   unsigned mask_;
   int year_, month_;

 public:
   Pass(Zone a, Zone b, int y, int m)
      : mask_((1u << static_cast<int>(a)) | (1u << static_cast<int>(b))), year_(y),
        month_(m)
   {
   }
   bool covers(const Journey &j) const
   {
      return j.year == year_ && j.month == month_ &&
            (mask_ >> static_cast<int>(j.farthest) & 1u);
   }
```

```
};

class SmartCard
{
   long long balance_;
   const Pass *pass_ = nullptr;

 public:
   explicit SmartCard(long long b) : balance_(b) {}
   void attach(const Pass *p) { pass_ = p; }
   long long balance() const { return balance_; }
   // returns fare charged, or -1 if denied
   long long travel(const Journey &j)
   {
      if (pass_ && pass_->covers(j)) return 0; // unlimited journey, nothing deducted
      long long f = fare(j);
      if (balance_ - f < MIN_BALANCE_PAISE) return -1;
      balance_ -= f;
      return f;
   }
};

static std::string rs(long long p)
{
   std::string s = std::to_string(p / 100) + ".";
   if (p % 100 < 10) s += "0";
   return s + std::to_string(p % 100);
}
static int failures = 0;
static void check(const char *name, long long expected, long long actual)
{
   bool ok = expected == actual;
   if (!ok) ++failures;
   std::cout << std::left << std::setw(46) << name << " expected " << std::setw(5)
             << expected << " actual " << std::setw(5) << actual
             << (ok ? " PASS" : " FAIL") << '\n';
}
int main()
{
   check("Airport weekday 12:00", 6000,
         fare({Zone::Airport, 12, 0, 1, false, 2026, 11}));
    check("Airport weekday 09:00 (peak +20%)", 7200,
         fare({Zone::Airport, 9, 0, 1, false, 2026, 11}));
   check("Inner Saturday 12:00 (-10%)", 900,
         fare({Zone::Inner, 12, 0, 5, false, 2026, 11}));
   check("Inner Saturday 09:00 (peak window)", 900,
         fare({Zone::Inner, 9, 0, 5, false, 2026, 11}));
   check("Outer Sunday 18:00 (no surcharge)", 2700,
         fare({Zone::Outer, 18, 0, 6, false, 2026, 11}));
   check("Airport Sunday student", 2700,
         fare({Zone::Airport, 12, 0, 6, true, 2026, 11}));
   check("Middle Friday 17:00 senior peak", 1200,
         fare({Zone::Middle, 17, 0, 4, true, 2026, 11}));
```

```
    Pass p(Zone::Inner, Zone::Middle, 2026, 11);
    SmartCard a(10000);
    a.attach(&p);
    check("Pass: Middle journey free", 0,
          a.travel({Zone::Middle, 9, 0, 1, false, 2026, 11}));
    check("Pass: balance unchanged", 10000, a.balance());
    check("Pass: Outer not covered (peak 3600)", 3600,
          a.travel({Zone::Outer, 9, 0, 1, false, 2026, 11}));
    check("Pass: balance after Outer", 6400, a.balance());
    SmartCard c(10000);
    c.attach(&p);
    check("Pass: next month not covered (fare 2000)", 2000,
          c.travel({Zone::Middle, 12, 0, 1, false, 2026, 12}));
    SmartCard b(10000);
    check("No pass: Airport peak denied (-1)", -1,
          b.travel({Zone::Airport, 9, 0, 1, false, 2026, 11}));
    std::cout << (failures == 0 ? "ALL PASSED" : "SOME FAILED") << '\n';
    std::cout << "Worked example: Airport, Saturday 09:00, student = Rs "
              << rs(fare({Zone::Airport, 9, 0, 5, true, 2026, 11})) << '\n';
}
```

Compiled with g++ -std=c++17 -Wall -Wextra t12.cpp -o t12; no warnings.

Output of ./t12:

```
Airport weekday 12:00                    expected 6000  actual 6000  PASS
Airport weekday 09:00 (peak +20%)        expected 7200  actual 7200  PASS
Inner Saturday 12:00 (-10%)              expected 900   actual 900   PASS
Inner Saturday 09:00 (peak window)       expected 900   actual 900   PASS
Outer Sunday 18:00 (no surcharge)        expected 2700  actual 2700  PASS
Airport Sunday student                   expected 2700  actual 2700  PASS
Middle Friday 17:00 senior peak          expected 1200  actual 1200  PASS
Pass: Middle journey free                expected 0     actual 0     PASS
Pass: balance unchanged                  expected 10000 actual 10000 PASS
Pass: Outer not covered (peak 3600)      expected 3600  actual 3600  PASS
Pass: balance after Outer                expected 6400  actual 6400  PASS
Pass: next month not covered (fare 2000) expected 2000  actual 2000  PASS
No pass: Airport peak denied (-1)        expected -1    actual -1    PASS
ALL PASSED
Worked example: Airport, Saturday 09:00, student = Rs 27.00
```

The three examples now work: Airport weekday off-peak Rs 60.00; Airport Saturday 09:00 student Rs 27.00; a Middle journey with an Inner + Middle pass is free and the balance stays 10,000 paise. I worked out the expected values by hand first (e.g. Airport weekday 09:00 = 6000 + 20% = 7200).

D. **Evidence-Based Defence**

Trade-offs. Keeping the tested core kept all 14 earlier tests valid. The cost is a longer fare() with an if / else-if chain, and a Journey struct every caller must fill correctly. A full redesign with a polymorphic DiscountRule hierarchy (Task 1) would be more extensible but adds classes and virtual calls that three small changes do not need.

Performance. The revised fare is still O(1) (one lookup, at most three percentage calculations, a bit-mask pass test). Per-card memory grows by one pointer (8 → 16 bytes). Totals stay long long, so the Task 10 total is still exact.

Risks accepted. (i) A6–A9 must be confirmed; each is one constant or one if. (ii) The pass is held by pointer, so it must outlive the card; a real system would store it inside the card record.